

Comparisons of Convolutional Neural Networks with Bayesian Program Learning for 20-Way One-Shot Classification task on Omniglot Dataset

Turgay Yıldız
Cognitive Science, Informatics Institute,
Middle East Technical University (METU)
Ankara, Turkey
yildiz.turgay@metu.edu.tr

Abstract—This paper aims to evaluate Convolutional Neural Networks (CNNs) against Bayesian Program Learning (BPL), a generative model that uses conditional probability. BPL incorporates concepts like *intuitive physics*, *intuitive psychology*, *compositionality*, *causality*, and *the learning-to-learn*. These concepts are considered potential avenues toward achieving human-level intelligence or artificial general intelligence (AGI). The study seeks to determine if CNNs can mimic BPL's use of *compositionality* or if they can compete with BPL without employing *compositionality*.

Index Terms—20-way, one-shot, omniglot, convolutional neural networks, bayesian program learning, human-level intelligence, artificial general intelligence, intuitive physics, intuitive psychology, causality, compositionality, learning-to-learn

I. INTRODUCTION

This project investigates whether Convolutional Neural Networks (CNNs) can be used to parse characters from the Omniglot dataset into their strokes, similar to how Bayesian Program Learning (BPL) operates. The aim is to explore the application of *compositionality* in CNNs, as there appears to be no alternative method to integrate this concept into CNNs. Additionally, this study compares the performance of various CNNs with other models cited in the literature and a baseline (Modified Hausdorff Distance) without the use of *compositionality*. Two main questions guide this research: First, can CNNs function effectively as parsers and, if so, can their performance rival that of BPL, which is known for incorporating key components of human-level intelligence such as *intuitive physics*, *intuitive psychology*, *causality*, *compositionality*, and *learning-to-learn*. The study also examines whether the concept of *compositionality* can be feasibly implemented in CNN architectures.

Secondly, the project explores whether Convolutional Neural Networks (CNNs) can be developed to match the performance of Bayesian Program Learning (BPL), which has surpassed human-level accuracy in 20-way one-shot classification on the Omniglot dataset. This comparison will be conducted without incorporating the aforementioned cognitive concepts.

These cognitive concepts have been extensively studied and applied in models within Cognitive Science research. The ultimate goal of such studies is to replicate human perception, learning, and cognitive processes within machine systems, essentially aiming to create machines that can [see](#), [learn](#), and [think like humans](#). Therefore, my approach goes beyond mere pattern recognition and emphasizes the importance of developing comprehensive models.

II. LITERATURE REVIEW

1) *Intuitive Physics*: Intuitive Physics can be conceptualized as an understanding of physical concepts like object permanence ([Lake et al., 2017](#)), enabling infants to comprehend the persistence of objects over time, even when they are not visually perceptible. This cognitive ability encompasses the recognition that objects retain their existence despite being temporarily occluded from view.

Furthermore, infants also discount physically impossible trajectories ([Lake et al., 2017](#)). They exhibit an awareness of how the behaviors or trajectories of objects conform to fundamental principles of physics, such as gravitational forces and friction. Additionally, infants demonstrate a comprehension of properties like solidity ([Lake et al., 2017](#)) and analogous characteristics.

Intuitive physics can be used as a tool to facilitate transfer learning in Deep Learning or to create models to have a more causally proper understanding of the world. To simulate the objects' behavior and trajectories under different conditions, [Battaglia et al. \(2013\)](#) used Intuitive Physics Engine (IPE) which is a probabilistic simulation tool. Physical Intuition also has a spatial and temporal facet. For example, Omniglot which is the dataset this paper employed was used by [Li et al. \(2021\)](#) to explore the spatio-temporal characterization capabilities of spiking neural networks (SNNs) by employing few-shot learning. Therefore, spatial and temporal aspects of data is a very important component of building models, especially models designed to reach to the human-level intelligence. Hence, one-shot and few-shot learning seem to

be the best way for this challenge.

2) *Intuitive Psychology*: Intuitive Psychology as mentioned by Lake et al. (2017) pertains to infants' capacity to attribute mental states, such as beliefs, goals, and desires, to other individuals. This cognitive ability significantly influences their learning processes and predictive reasoning. For instance, Lake et al. (2017) gives a video game example : when observing a person engage in a video game, a child can infer the intentions behind the player's actions based on their understanding of psychological states. Consequently, the child may discern certain in-game objects as detrimental, prompting them to avoid interaction, while others are perceived as advantageous, thereby motivating pursuit. This comprehension of psychological states facilitates infants' social cognition and informs their decision-making in various contexts. Ability to differentiate between what is animate and what is inanimate may ease the process of learning. We assume that human beings are rational, and we expect them to behave in predicted ways. We know that inanimate objects are not rational and only subject to the physical rules. These concepts facilitate our understanding or modeling of the reality. As Lake et al. (2017) put forward, model building is the hallmark of human-level learning. That is where Cognitive Science can make great contributions to the AI, by bridging the gap between machines and people.

3) *Causality*: As mentioned above, both intuitive physics and intuitive psychology revolve around the construction of models. As previously noted, these elements are often lacking in conventional AI and DL techniques. Human cognition inherently involves an ongoing endeavor to construct representations of the surrounding world. This novel approach lies at the intersection of AI and Cognitive Science. Notably, as Lake et al. (2017) claimed, both intuitive physics and intuitive psychology can be seen as causal models of reality.

Lake et al. (2015) used Bayesian Program Learning (BPL) to capture the causal and compositional properties of the Omniglot dataset. By giving image caption examples from Deep Neural Networks, Lake et al. (2017) shows that while Deep Learning models excel in pattern and object recognition, they fall short in establishing causal relationships among these entities. This deficiency underscores a significant disparity between machine intelligence and human-level cognition. Primarily, it stems from the absence of key concepts such as intuitive understanding of physical phenomena and the causative links between them. Consequently, bridging this gap necessitates a paradigm shift towards integrating causal reasoning frameworks into AI and ML methodologies.

4) *Compositionality*: In this project, compositionality will serve as a pivotal implementation method. Typically utilized in algorithms such as one-shot learning and few-shot learning. Compositionality stands as a technique enabling models to learn from sparse data (Lake et al., 2017). This approach

is closely intertwined with learning-to-learn, also known as transfer learning or meta-learning (Lake et al., 2011). Notably, compositionality and learning-to-learn are intricately interconnected concepts. The essence of compositionality lies in the notion that learning from individual components of a system can expedite the learning of a different system with similar components as a whole, particularly in scenarios where data availability is limited. By initially acquiring knowledge of constituent elements, subsequent learning endeavors can capitalize on this foundation. Thereby, this can facilitate more efficient learning processes with minimal data input. Indeed, the ability to learn from sparse datasets represents a paramount challenge in the pursuit of developing machines with human-like cognitive capacities. As such, compositionality emerges as a crucial tool in the quest to bridge this disparity between artificial and human intelligence.

5) *Learning-to-learn*: Learning-to-learn, also referred to as transfer learning or meta-learning, encompasses a nuanced distinction depending on the context in which it is employed. Within the realm of Deep Learning, transfer learning or meta-learning typically involves the utilization of pre-trained layers to expedite the learning process for subsequent tasks. However, in the domain of Cognitive Science, learning-to-learn embodies a more profound connotation aligned with its literal interpretation. Consider a child's acquisition of knowledge regarding the *composition* of letters or digits, recognizing them as distinct arrangements of discrete shapes. Beyond mere task-specific application, this cognitive process engenders a fundamental understanding that entities can be comprised of constituent components, thereby facilitating discrimination and comprehension. This form of learning transcends pattern recognition, engendering the assimilation of novel concepts and the construction of cognitive models. Consequently, individuals develop the capacity to learn how to learn, a trait embodied by their adeptness at deriving insights from sparse data. The efficacy of learning-to-learn lies in its facilitation of not only task-specific adaptation but also the acquisition of generalizable cognitive frameworks. By embracing the underlying principles of compositional learning, individuals become adept at extrapolating knowledge across diverse domains, thereby enhancing their capacity for rapid and efficient learning. Thus, learning-to-learn emerges as a cornerstone in the endeavor to equip artificial systems with human-like cognitive capabilities, enabling them to thrive in dynamic and data-scarce environments.

III. WHAT'S NOVEL

First and foremost, the project will initiate with a comprehensive review of prior research in the field to clarify the existing body of knowledge. This critical review will include an examination of how foundational concepts such as causality, compositionality, and learning-to-learn have been utilized within the one-shot and few-shot learning paradigms. The motivation for this exploration stems from

the recognition of these cognitive principles as key elements that may distinguish human learning processes from those of machines, particularly in the context of model building versus pattern recognition. Consequently, the ability to learn from sparse data becomes a crucial focus, representing a measure of the differences between human and machine learning capabilities and serving as a potential indicator of progress toward achieving human-level intelligence.

The unique aspect of this project, distinct from previous studies, is its innovative use of deep learning models to classify handwritten characters, specifically by leveraging the principle of compositionality. While prior research, like that by Lake et al. (2017), has showcased the effectiveness of Bayesian Program Learning (BPL) in achieving human-level performance in one-shot classification tasks, this project takes a different approach. Instead of utilizing BPL, it focuses on employing the concept of *compositionality* through various data processing techniques and the application of deep learning models. This deviation seeks to explore new possibilities in model performance and learning capabilities.

This project aims to shift away from the statistical and conditional probability approaches inherent in Bayesian Program Learning (BPL), focusing instead on conceptualizing and exploiting compositionality as the foundational element of its algorithmic framework. This approach requires a creative rethinking of how compositionality can be transformed into a practical algorithmic application. Unlike BPL, which inherently uses a causal and compositional model, this project seeks to distill the essence of compositionality and integrate it into the learning process without relying on probabilistic methods.

Lake et al. (2011) created a library of strokes. They clustered 40,000 strokes by using k-means to form 1000 centroids. The library is composed of these 1000 centroids. Then, they tested three different models: the stroke model which in fact is a generative model that they introduced, the Deep Boltzman Machine (DBM), and Nearest Neighbor(NN) for 20 way classification from one example. The stroke model achieved a performance of 54.9%, significantly outperforming the DBM and NN, which scored 39.6% and 15.7%, respectively.

Li et al. (2021) transformed Omniglot data into videos by incorporating spatial and temporal information from text files, reconstructing the record of strokes into a video format that depicts the writing process. They employed a linear interpolation algorithm to enhance the temporal data's accuracy for these reconstructions. While I plan to employ a similar technique, my approach will differ as I will not utilize the time dimension; thus, no interpolation method is necessary. Instead, I will solely use the spatial data from the text files to construct character components (strokes).

The innovation of this project lies in its aim to pioneer new directions in Artificial Intelligence (AI) and Deep Learning (DL) by harnessing the principles of *compositionality*. This will be achieved by employing a unique data processing technique and implementation approach, designed to enhance the performance of classifying handwritten characters from sparse data.

IV. DATASET:

Each process of implementation of the project is going to be clarified as step by step. Specifically, a convolutional neural network (Fig. 12) was trained to discern and learn components (subparts) of a dataset called Omniglot which composed of 1623 distinct handwritten characters from 50 different alphabets. The Omniglot dataset aims to cultivate algorithms that mimic human learning processes more closely. Each character was independently drawn by 20 individuals through Amazon's Mechanical Turk platform. Each image is paired with stroke data, presenting sequences of [x,y,t] coordinates where time (t) is measured in milliseconds.

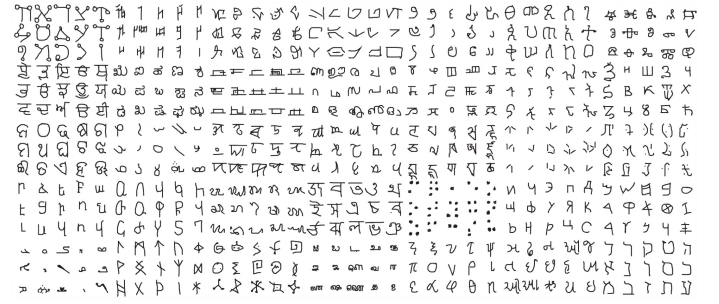


Fig. 1. Omniglot Dataset from Lake et al. (2015)

Note : To implement aforementioned concepts, the dataset is used mostly in generative models like BPL (Lake et al., 2011, 2015 and 2017), and in those models, BPL was used as a parser and as a generative model. In other papers, it was generally used for its spatial and temporal aspect (Li et al., 2021) and without employing aforementioned concepts.

The dataset is divided into two parts: the first part, consisting of 30 alphabets, serves as the **pre-training dataset**, while the second part, comprising the remaining 20 alphabets, can be designated as the **one-shot dataset**. In the literature, such as the work by Lake et al. (2015), the first dataset is used to pre-train BPL and various deep learning models. These models are then trained with a single image from each class (character) from the one-shot dataset, without exposure to other images. Subsequently, all models are tested for one-shot classification on the one-shot dataset, which they were not trained on.

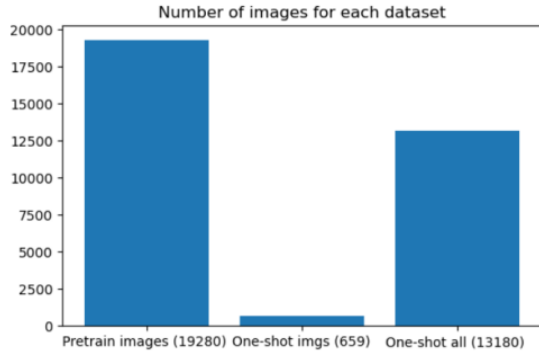


Fig. 2. Number of images for pre-training and one-shot classification.

The dataset is structured into two segments: the first part is used for pretraining the models, and the second part is designated for one-shot learning. This one-shot dataset permits the use of only 659 images because it contains 659 characters spread across 20 alphabets. Each character has 20 distinct images, and by selecting one image from each character, the total comes to 659 images.

V. FIRST PART OF THE STUDY: APPLICATION OF COMPOSITIONALITY TO CNN

A. Preprocessing

Each image in the dataset is 105 by 105 pixels. Images can be read by Python as grayscale and converted into binary values, where each pixel is represented as either zero or one. Consequently, there is no need for preprocessing these images, as PyTorch offers transformation methods that can normalize and augment the images in various ways. However, the stroke data, which is stored in text files, presents a more challenging processing task. These text files can be utilized in various ways, such as in generative models like Bayesian Program Learning, as noted in studies by [Lake et al. \(2015 and 2017\)](#), or in approaches like spatio-temporal sparse few-shot learning as previously mentioned.

Despite these existing methods, this project will diverge from them for two main reasons. First, it will employ deep learning techniques instead of Bayesian Program Learning. Second, the project is committed to strictly adhering to the principles of Cognitive Science, thus excluding the second approach involving spatio-temporal data usage. To address these considerations, I have developed a new method to process this data. This project is innovative both in terms of the data processing techniques and its implementation, details of which will be elaborated upon shortly.

B. Text Files

The text files containing stroke data are structured with specific formatting cues. Each file begins with the word "START" and comprises three columns per row. The first two columns represent the trajectories of strokes in the x-y

coordinate system, and the third column corresponds to the time axis. These values continue for a number of rows until the stroke ends, which is indicated by the word "BREAK." This signifies that one stroke has been completed and the next will begin. The rows then continue in this pattern until another "BREAK" occurs. All text files follow this structure, starting with "START" and continuing with many "BREAK"s. At the end of the files, there is the word "END".

```
START
11.745075, -15.097315, 0
13.677961, -14.130872, 139
14.644404, -14.130872, 147
.
.
.
.
45.66777, -34.43333, 345
BREAK
.
```

This is first stroke.

Fig. 3. An image to depict the data inside the txt files.

0709_01.txt

0709_02.txt

0709_03.txt

0709_04.txt

0709_05.txt

0709_06.txt

0709_07.txt

0709_08.txt

```
1 START
2 26.697628, -31.228947, 0
3 26.697628, -32.908789, 80
4 26.697628, -33.74871, 104
5 26.697628, -34.588631, 120
6 26.697628, -35.428552, 136
7 26.697628, -36.268473, 144
8 26.697628, -37.108394, 152
9 26.697628, -37.948315, 160
10 26.697628, -38.788236, 168
11 26.697628, -39.628157, 176
12 26.697628, -40.468078, 184
13 26.697628, -41.307999, 192
14 25.857708, -42.14792, 216
15 25.857708, -42.987841, 240
16 BREAK
```

Fig. 4. These images show some of the txt files' names and content of one of them.

This is the first character from 'Early Aramaic' alphabet.



Fig. 5. First character of Early Aramaic Alphabet.

When it is plotted with stroke data :



Fig. 6. First character of Early Aramaic Alphabet when plotted with stroke order.

However, employing causality in this project faces challenges and thus requires a different approach to data processing. The primary reason is the variability in the number of strokes across different characters; some characters may have as few as three strokes, while others could have up to fifteen. Furthermore, if a fixed dimension of 10 strokes is chosen for uniformity, the absence of information in characters with fewer than 10 strokes would result in non-representative data. Therefore, due to these constraints, it is not feasible to directly employ causality based on stroke order in this project’s current setup.

21

The images I created from the txt file data are in black and white because we only require one channel, not the full RGB spectrum. Each image maintains a dimension of 105 by 105 pixels, identical to the original input images.

 λT
$$v - l$$

Note: The third stroke is empty, so its corresponding image does not contain any '1' values.

					
1	2	3			
					
	8			11	

As you can see, some of the strokes do not contain any information. Therefore, I arranged them in the following order based on the amount of information each contains, prioritizing those with the most significant content to those with the least or none. This method ensures that the most information-rich strokes are used for constructing the ground truth images.

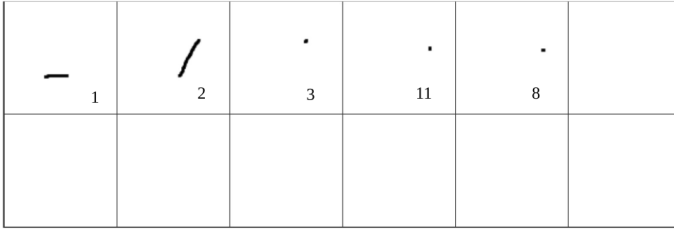


Fig. 11. When they were put in order

I selected the first five of the most useful strokes based on their information content. In this instance, there were exactly five useful strokes, allowing us to utilize all of them. However, in cases where there are more than five informative strokes, some data may inevitably be lost when we limit the selection to the top five. While this method may result in the loss of some information, it typically does not significantly impact the overall representation, ensuring that the most critical features are still captured in the ground truth images.

C. A U-NET like Network:

This model (Fig. 12) was designed to function as a parser so as to employ *compositionality*, facilitating comparisons with Bayesian Program Learning (BPL). This was the first aim of this project as mentioned before. This model processes images that have been structured into 5 dimensions. This dimensional configuration is tailored to analyze the compositionality within the images, optimizing the network's ability to interpret and reconstruct the underlying patterns of strokes effectively. Such a design was thought to allow the U-NET like network to handle complex parsing tasks that mimic the capabilities of BPL but with a distinct methodological approach.

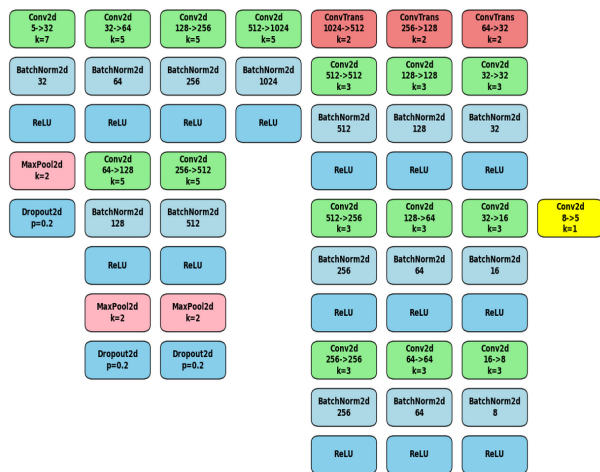


Fig. 12. The architecture of U-NET like network

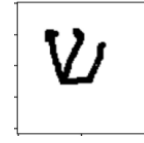


Fig. 13. An input for U-NET like network. This is 5-dimensional and all dimensions have the same data

The U-NET like network employs a Mean Squared Error (MSE) loss function to calculate the loss. This function measures the difference between the network's outputs—5-dimensional images each sized at 105 by 105 pixels—and the corresponding ground truth images. By using MSE, the network quantifies the accuracy of its predictions, ensuring that the parsed output closely matches the detailed structure and composition of the ground truth images. This comparison is crucial for refining the model's performance and enhancing its parsing capabilities.

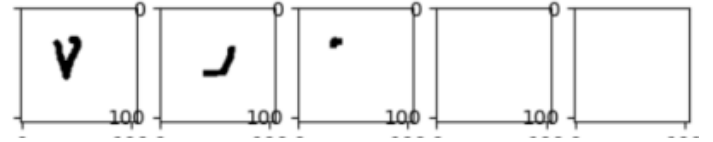


Fig. 14. The ground truth which has 5 dimensions and has an order.

If we were to compare one of the output images with its corresponding ground truth, we would look at how closely the neural network's output matches the expected result :

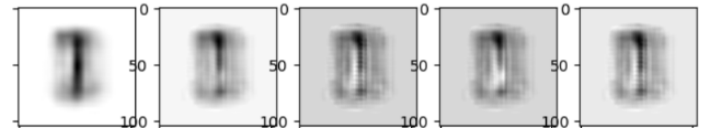


Fig. 15. The output of the U-NET like network.

The ground truth:

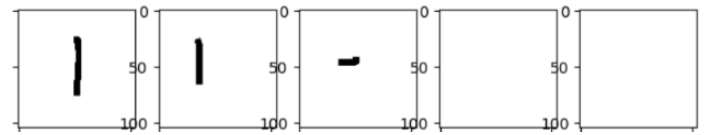


Fig. 16. The corresponding ground truth.

As demonstrated by the images above, even though the U-NET like CNN is capable of learning relationships between inputs and outputs, it falls short in parsing the characters into their individual strokes. Consequently, this model fails to function effectively as a parser, similar to the capabilities demonstrated by Bayesian Program Learning (BPL) when employing compositionality. This inability leads to the conclusion that a CNN, in this configuration, cannot be used as a parser like BPL. Thus, the results from this U-NET like model will not be compared with other models. Moving

forward, the project shifts focus towards developing more robust CNN architectures, as previously mentioned. This effort forms the second primary objective of this project, aiming to construct CNNs that can be more directly compared with BPL.

From now on, I will focus on comparing the classification capabilities of CNNs and BPL without considering compositionality. This approach will evaluate the fundamental performance of these models in classification tasks, allowing for a direct comparison of their effectiveness in handling data and making accurate predictions, independent of their ability to interpret or utilize compositional structures in the data.

VI. SECOND PART OF THE STUDY: COMPARISON BETWEEN CNNs AND BPL WITHOUT EMPLOYING COMPOSITIONALITY

A. Hyper-parameter Tunings :

1) **Initial Learning Rate:** I conducted hyper-parameter tuning for the first CNN (Fig. 21, and 22) which includes convolutional, batch normalization, ReLU, and fully connected layers. It was developed from scratch. The results of this tuning process are detailed below. This step was crucial for optimizing the performance of the CNN, ensuring that it operates efficiently and accurately under the designated task conditions. Adjusting parameters such as learning rate, batch size, loss functions and optimizers allowed for a tailored approach that maximizes the model's effectiveness in classification.

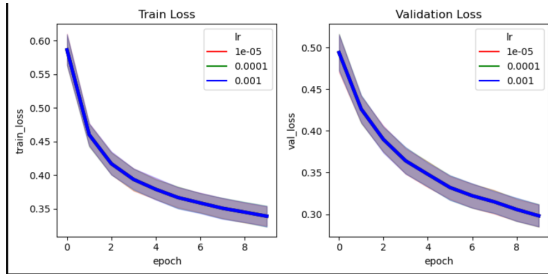


Fig. 17. Comparisons of learning rates with respect to train and validation losses

As you can see, when we compare the learning rates, the results are very close to each other. However, I chose 0.01. Since I always use a learning rate scheduler, the model's learning eventually stabilizes. Unless the model is very deep, I do not prefer to use small learning rates.

2) **Step Learning Rate Scheduler:** If the initial learning rate is too high, it may cause oscillations; if it is too low, it may take too long to converge. Therefore, I initially chose a high learning rate with a rapidly decreasing scheduler. Then, I examined the losses to determine when the model stopped oscillating. However, the initial learning rate (0.01) did not

cause any oscillations. Finally, I ran the model with this initial learning rate using a slowly decreasing scheduler.

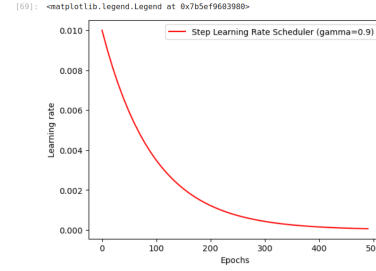


Fig. 18. Step Learning Rate Scheduler

3) **Loss Functions:** First of all, you may notice something (Fig. 19): accuracies do not change throughout the epochs. This is because I calculated only the final accuracies at the end of the last epoch. That is why you see a horizontal line instead of curves.

Secondly, we cannot directly compare the Binary Cross-Entropy with logits (BCE with logits) and Mean Squared Error (MSE) loss functions based on their numerical loss values because they operate on different scales and metrics. These differences mean that loss values from each method aren't directly comparable. However, when evaluating the training and validation accuracies, it appears that the BCE with logits loss function performs better than MSE. Based on this comparative advantage in accuracy, I have chosen to use the BCE with logits loss function for this model. This decision is aimed at optimizing the model's performance in classification tasks.

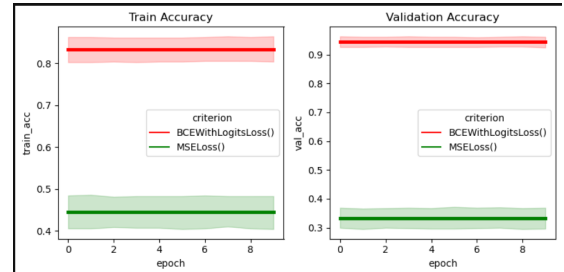


Fig. 19. Comparisons of loss functions with respect to accuracies

4) **Optimizers:** I also conducted comparisons between different optimizers, as detailed below. This comparison was essential to determine which optimizer would enhance the model's learning efficiency and overall performance. By testing various optimizers, such as Adam, and RMSprop, I was able to assess their impact on the model's convergence. The results of these comparisons guided the selection of the most effective optimizer for the task at hand.

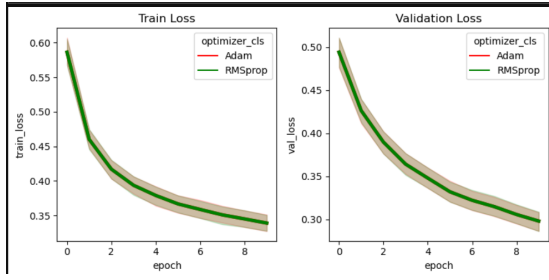


Fig. 20. Comparisons of optimizers with respect to losses

It appears that there is no significant difference in performance between the Adam and RMSprop optimizers based on the tests conducted. However, I have chosen to use Adam for this model. This decision may be influenced by factors such as Adam's well-documented effectiveness in various scenarios, and its efficiency. Adam is generally known for its robust performance across a wide range of deep learning applications, making it a reliable choice for optimizing the model.

B. Three Different Models:

First of all, since this second part of the project does not require any preprocessing, we will skip that step. The models, which will be explained soon, were first trained with the pre-trained data as mentioned earlier. Remember, the first part of the dataset is composed of 30 alphabets, and the second part consists of the remaining 20 alphabets. Therefore, these models were also trained using only one image from classes (characters) of the second part of the data (one-shot data). Then, they were employed for the task of 20-way one-shot classification on the one-shot data.

I employed the same hyper-parameters for both ResNet-18 and ResNet-50 models. We will now examine three distinct models: a CNN that was developed from scratch, as previously mentioned, along with ResNet-18 and ResNet-50. For these two pretrained networks, I froze the first 6 blocks (children) but left the last 4 blocks active for further training. Below (Fig. 21, and 22), you can see the architecture of the CNN.

layer	in_channels	out_channels	kernel_size	padding	activation	batch_norm	pooling	dropout
Conv2d	2	64	7	3	ReLU	True	MaxPool2d	0.1
Conv2d	64	128	5	2	ReLU	True	MaxPool2d	0.1
Conv2d	128	256	5	2	ReLU	True	MaxPool2d	0.1
Conv2d	256	512	5	2	ReLU	True	MaxPool2d	0.1

Fig. 21. Convolutional Layers

layer	in_features	out_features	activation	batch_norm	dropout
Linear	18432	1024	ReLU	True	0.1
Linear	1024	512	ReLU	True	nan
Linear	512	256	ReLU	True	0.1
Linear	256	128	ReLU	True	nan
Linear	128	64	ReLU	True	nan
Linear	64	1	nan	nan	nan

Fig. 22. Fully Connected Layers

I used Binary Cross Entropy Loss (with logits) for all models because this loss function is more numerically stable when models produce raw outputs. This choice was also influenced by the fact that I did not implement a sigmoid activation in the final layers of these networks. This is why I did not include the standard Binary Cross Entropy loss (without logits) in the hyper-parameter tuning, as it requires output to be passed through a sigmoid function to convert logits to probabilities, which was not compatible with the structure of my models.

Below (Fig. 23), you can see the loss metrics for each of these three models.

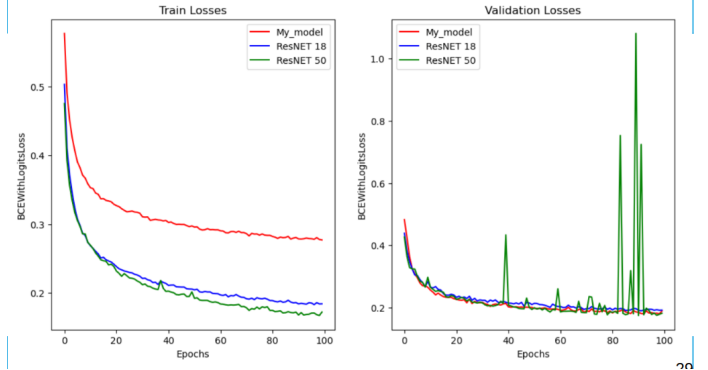


Fig. 23. Train and validation losses of three models : CNN, ResNET-18 and Res-Net 50

Firstly, as shown in the plot above (Fig. 23), the CNN developed from scratch displays the worst training losses compared to the pretrained ResNet models. However, it exhibits the lowest validation losses and, notably, outperforms both ResNet-18 and ResNet-50 in the 20-way one-shot classification task. This indicates that despite its higher training losses, the custom CNN may be more effectively generalizing to unseen data, a crucial aspect of model performance in practical scenarios.

Secondly, ResNet-50 started to oscillate between 80 and 90 epochs. The batch size (32) could be a reason for this

oscillation. Even if the data was shuffled, there may still be some outliers that batch normalization could not handle. The learning rate started at 0.01. However, if the learning rate were the problem, the model would probably oscillate at the beginning rather than at the end, due to the scheduler. Since ResNet-50 is the deepest model, it may have been most affected by weight updates, and when the model overfitted, the validation losses started to jump.

The subsequent plot (Fig. 24) presents the number of correct and incorrect predictions for two-way and twenty-way classifications. These results are specific to the CNN and cover both validation and one-shot data scenarios. This visual representation helps in understanding the model's predictive accuracy and robustness under different classification complexities.

This plot below (Fig. 24) shows **two-way** classification results: number of correct and wrong predictions and accuracy rates.

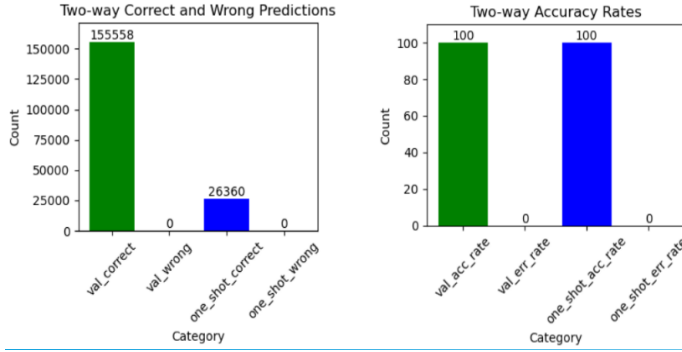


Fig. 24. Two-Way classification results: on validation data and One-Shot data

This plot below (Fig. 25) shows **twenty-way** classification results: number of correct and wrong predictions and accuracy rates.

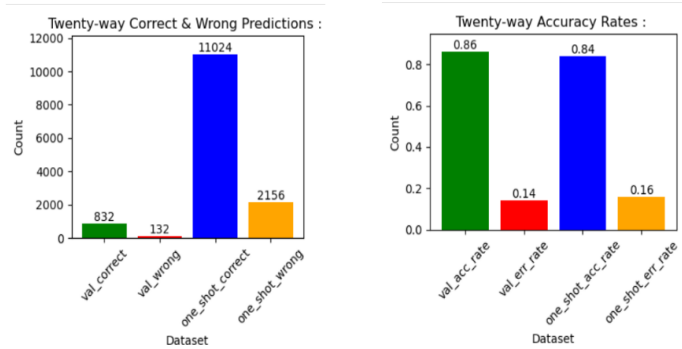


Fig. 25. Twenty-Way Classification results: on validation data and One-Shot data

At the conclusion of the analysis, the comparisons of all models are presented below (Fig. 26), arranged in order from the best performing model with the lowest error rate to the worst model, which serves as the baseline (Modified

Hausdorff Distance) with the highest error rate. Of these models, only three are my own creations; the remaining models are taken from the study conducted by [Lake et al. \(2015\)](#). This arrangement allows for a clear and direct comparison, showcasing the effectiveness of each model in terms of error rates, and highlighting the advancements or shortcomings relative to the baseline and other existing models.

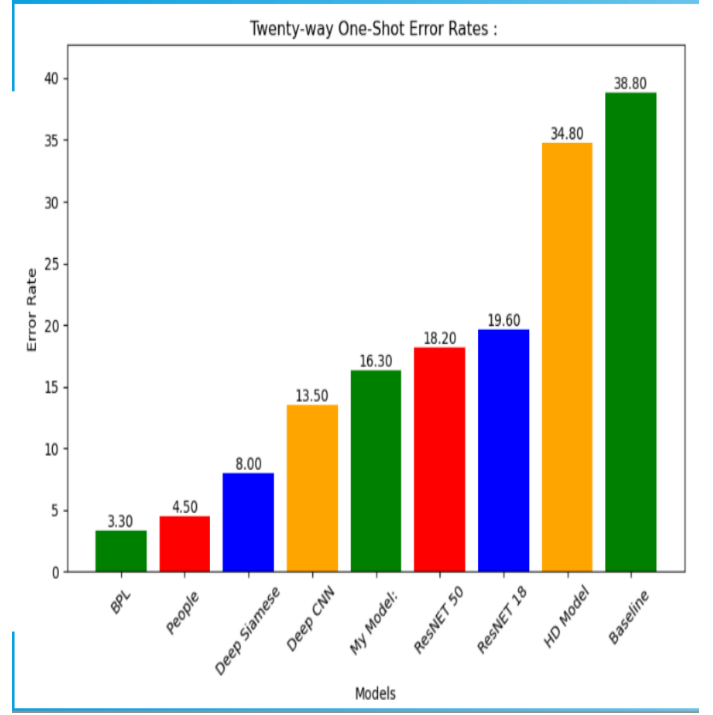


Fig. 26. Comparisons of all models.

VII. CONCLUSION AND DISCUSSION:

In this study, we aimed to evaluate and compare the capabilities of Convolutional Neural Networks (CNNs) and Bayesian Program Learning (BPL) in the context of *parsing* and *classifying* handwritten characters from the Omniglot dataset. Our approach was divided into several key steps, each contributing to a comprehensive understanding of the models' strengths and weaknesses.

Initially, we explored the concept of *compositionality* by attempting to parse characters into their constituent strokes using a U-NET like CNN. Despite the model's ability to learn relationships between inputs and outputs, it failed to effectively parse the characters, demonstrating that CNNs in this configuration cannot replicate the parsing capabilities of BPL.

Shifting focus, we then aimed to compare the classification power of CNNs and BPL without considering

compositionality. A custom CNN was developed and subjected to hyper-parameter tuning, alongside evaluations of various optimizers. The choice of the Binary Cross Entropy with logits loss function (BCEWithLogitsLoss) was driven by its numerical stability and compatibility with raw output values from the models.

Three different models were tested: a custom CNN, ResNet-18, and ResNet-50. The analysis of the hyper-parameter tuning results showed that different loss functions significantly influenced the model performance. The plots revealed that the custom CNN, despite having higher training losses, showed the lowest validation losses and outperformed both ResNet models in the 20-way one-shot classification task. This suggests that the custom CNN had better generalization capabilities on unseen data.

Further analysis of optimizer performance indicated that there was no significant difference between the Adam and RMSprop optimizers, though Adam was chosen for its robustness. Comparing BCEWithLogitsLoss to MSELoss revealed that BCEWithLogitsLoss resulted in better train and validation accuracies, which informed its selection for this study.

In two-way classification tasks, the custom CNN achieved perfect accuracy on both validation and one-shot data, demonstrating its efficacy in simpler classification scenarios. However, in the more complex twenty-way classification tasks, while not achieving perfect accuracy, the custom CNN still performed commendably, achieving 86% accuracy on validation data and 84% on one-shot data.

Comparing all models, including those from [Lake et al. \(2015\)](#), the custom CNN demonstrated competitive performance with a twenty-way one-shot error rate of 16.30%, outperforming ResNet-18 and ResNet-50, which had error rates of 19.60% and 18.20%, respectively.

A. Key Insights

- **Parsing Capabilities:** CNNs, as configured in this study, are not suitable for parsing characters into strokes akin to BPL, which excels in leveraging compositionality.
- **Classification Performance:** When compositionality is not considered, CNNs, particularly those custom-designed and well-tuned, can achieve impressive classification performance, sometimes surpassing established architectures like ResNet.
- **Hyper-parameter Tuning and Optimization:** The choice of loss functions significantly impacts model performance. BCEWithLogitsLoss and the Adam optimizer provided robust performance in this study.
- **Generalization:** The custom CNN’s ability to generalize well on unseen data highlights the importance of model design and training strategies tailored to the specific nature of the task.

Future work could explore further enhancements in model architectures, integration of compositionality in CNNs, and more extensive comparisons with other state-of-the-art models. This study serves as a foundation for advancing our understanding of the capabilities and limitations of CNNs in tasks traditionally dominated by generative models like BPL.

REFERENCES

- [1] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum (2015). “Human-level concept learning through probabilistic program induction.” *Science*, 350(6266), 1332-1338. DOI: [10.1126/science.aab3050](https://doi.org/10.1126/science.aab3050). Available at: <https://www.science.org/doi/abs/10.1126/science.aab3050>.
- [2] Brenden M. Lake, Ruslan Salakhutdinov, Jason Gross, and Joshua B. Tenenbaum (2011). “One shot learning of simple visual concepts.” *Proceedings of the Annual Meeting of the Cognitive Science Society*, 33. Available at: <https://escholarship.org/uc/item/4ht821jx>.
- [3] Peter W. Battaglia, Jessica B. Hamrick, and Joshua B. Tenenbaum (2013). “Simulation as an engine of physical scene understanding.” *Proceedings of the National Academy of Sciences*, 110(45), 18327-18332. DOI: [10.1073/pnas.1306572110](https://doi.org/10.1073/pnas.1306572110). Available at: <https://www.pnas.org/doi/abs/10.1073/pnas.1306572110>.
- [4] Brenden M. Lake, Tomer D. Ullman, Joshua B. Tenenbaum, and Samuel J. Gershman (2017). “Building Machines That Learn and Think Like People.” *Behavioral and Brain Sciences*, 40. DOI: [10.1017/s0140525x16001837](https://doi.org/10.1017/s0140525x16001837).
- [5] Yang Li, Yiting Dong, Dongcheng Zhao, and Yi Zeng (2021). “N-Omniglot: a Large-scale Neuromorphic Dataset for Spatio-Temporal Sparse Few-shot Learning.” *CoRR*, abs/2112.13230. Available at: <https://arxiv.org/abs/2112.13230>.